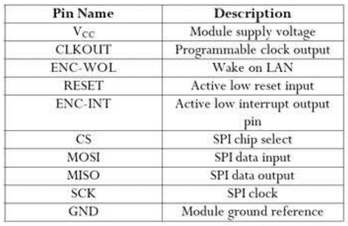
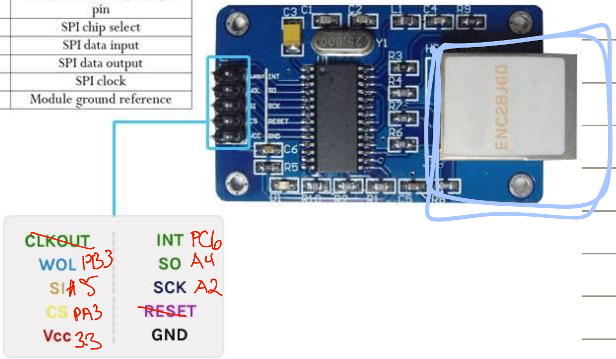
* Change HW address in provided code to work

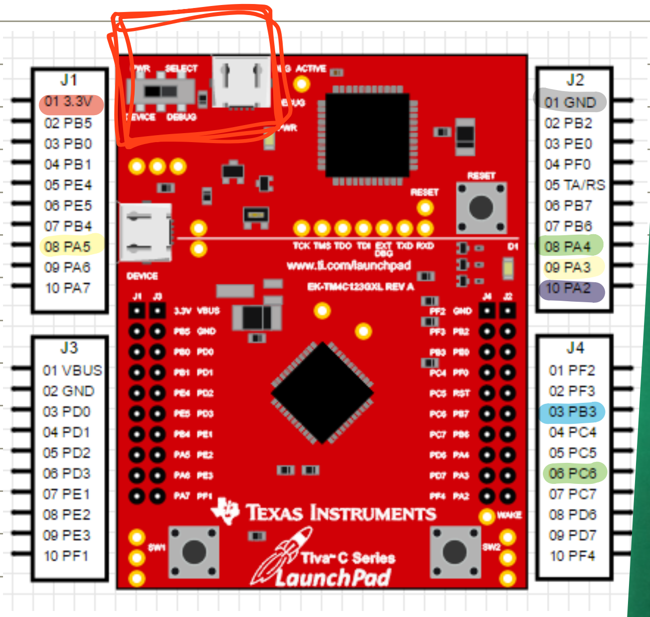
MOSI ↔ PA5
MISO ↔ PA4
SCLk PA2
~CS PA3
WOL PB3
INT PC6
Vdd 3.3
VSS 0V

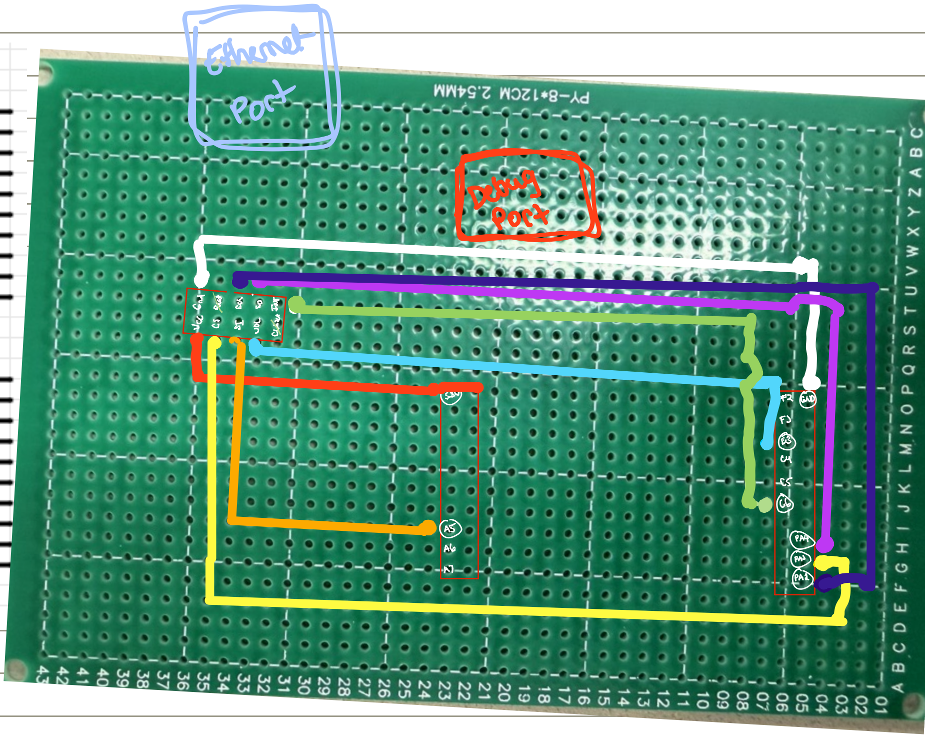
push buttons are volatile
avoids using cached variable



Pin Name	Description
V_{CC}	Module supply voltage
CLKOUT	Programmable clock output
ENC-WOL	Wake on LAN
RESET	Active low reset input
ENC-INT	Active low interrupt output pin
CS	SPI chip select
MOSI	SPI data input
MISO	SPI data output
SCK	SPI clock
GND	Module ground reference

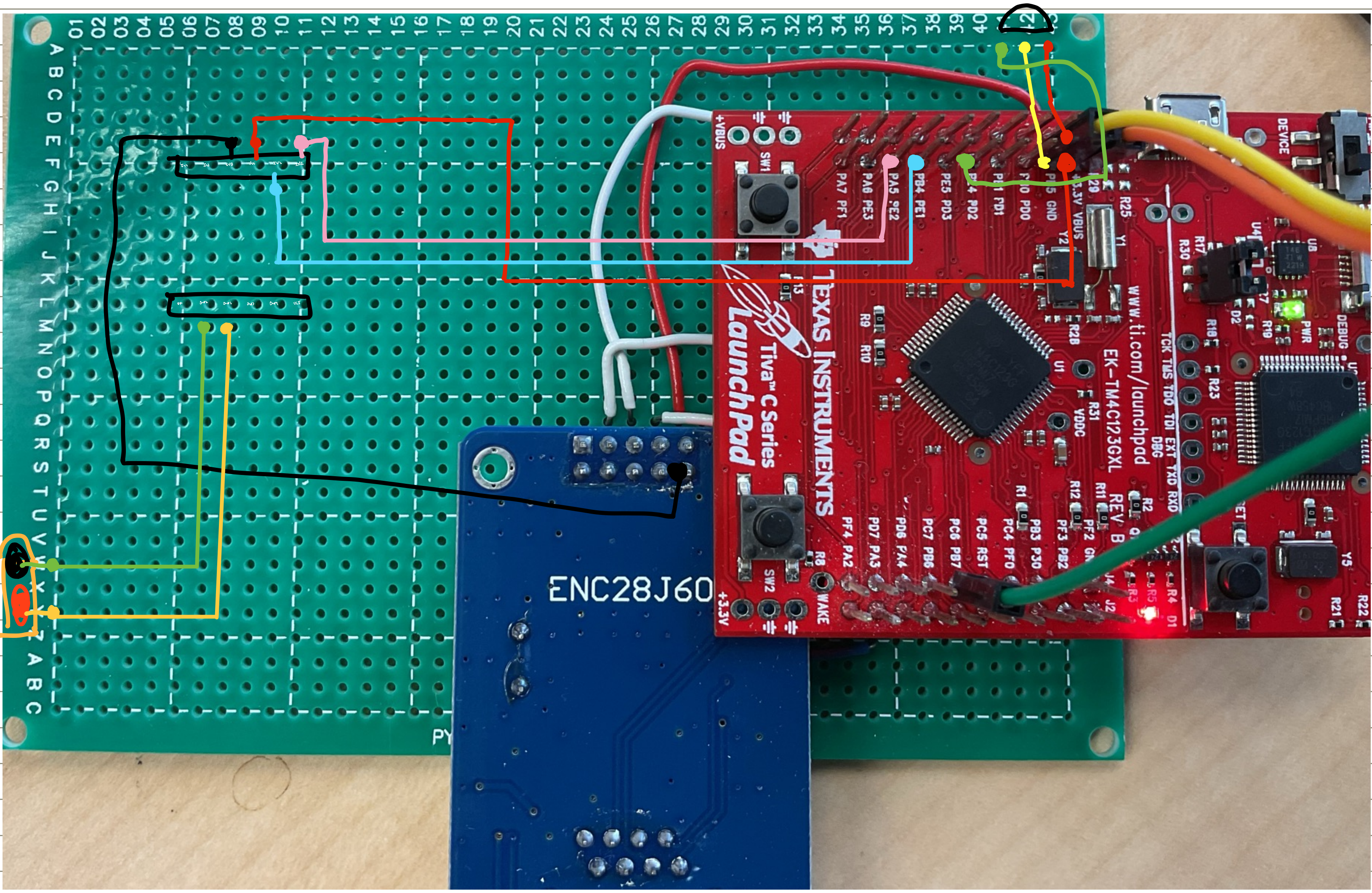






PE1 = IN1
PE2 = IN2

Out 1 : Motor Red
Out 2 : Motor black



- Offload IPV4 Checksum
  UDP
  TCP - final
  > to software (disable them)

control panel > local network...

- install ethtool
  somehow find Eth name

Wireshark: validate checksum

Syn sent needs timeout, event and time driven

window

TCP SM

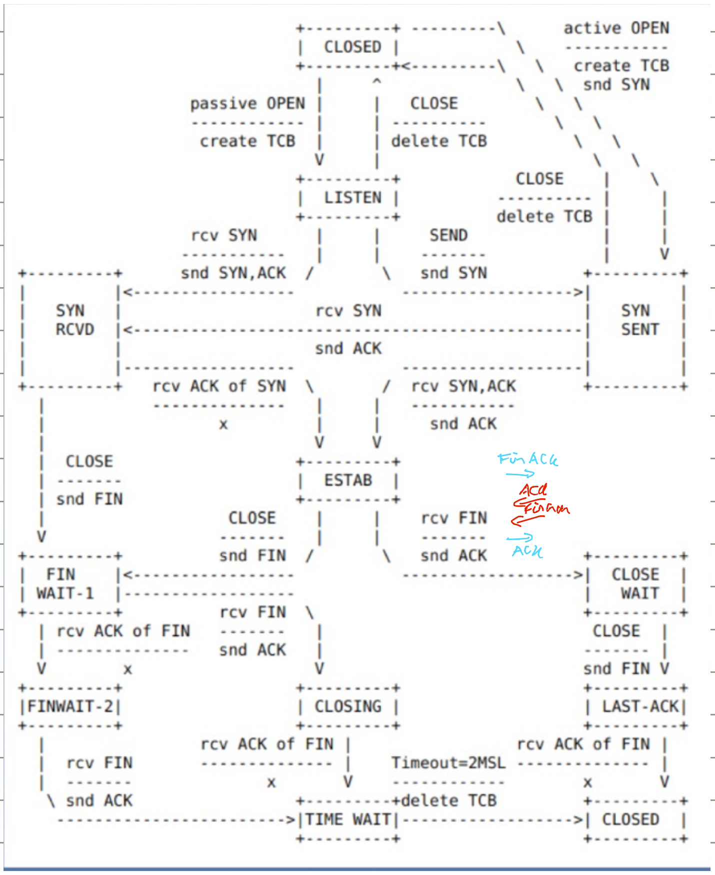
Active open:

Client | Server | Syn Ack
(state) Sync_sent —sync→ Listen | 0
←ack+sync— sync_received | 30 1
established (—ack→ established | 30

Socket | TCP packet

Ip.h has many functions, Mqtt

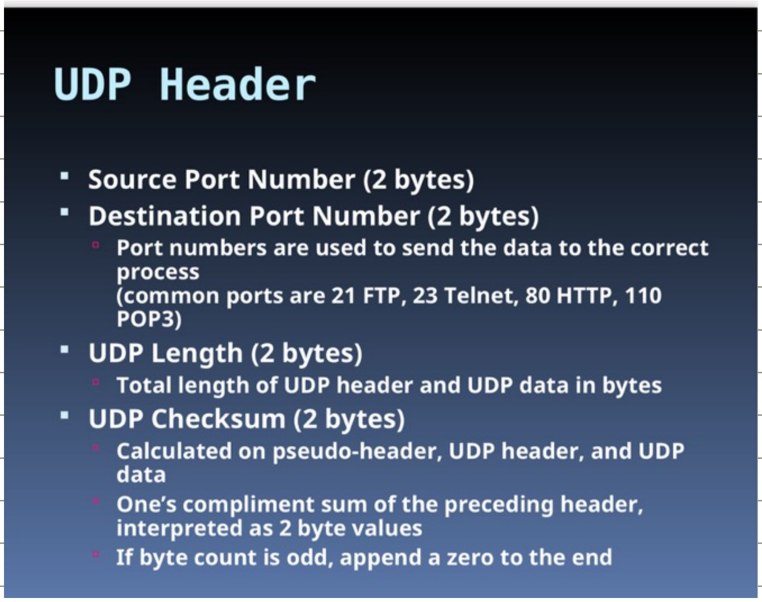
htons → reversed endianness of data



```
                              +---------+ ---------\      active OPEN
                              |  CLOSED |            \    -----------
                              +---------+<---------\   \   create TCB
                                |     ^              \   \  snd SYN
                   passive OPEN |     |   CLOSE        \   \
                   ------------ |     | ----------       \   \
                    create TCB  |     | delete TCB         \   \
                                V     |                      \   \
                              +---------+            CLOSE    |    \
                              |  LISTEN |          ---------- |     |
                              +---------+          delete TCB |     |
                   rcv SYN      |     |     SEND              |     |
                  -----------   |     |    -------            |     V
 +---------+      snd SYN,ACK  /       \   snd SYN          +---------+
 |         |<-----------------           ------------------>|         |
 |   SYN   |                    rcv SYN                     |   SYN   |
 |   RCVD  |<-----------------------------------------------|   SENT  |
 |         |                    snd ACK                     |         |
 |         |------------------           -------------------|         |
 +---------+   rcv ACK of SYN  \       /  rcv SYN,ACK       +---------+
   |           --------------   |     |   -----------
   |                  x         |     |     snd ACK
   |                            V     V
   |  CLOSE                   +---------+
   | -------                  |  ESTAB  |
   | snd FIN                  +---------+
   |                   CLOSE    |     |    rcv FIN
   V                  -------   |     |    -------
 +---------+          snd FIN  /       \   snd ACK          +---------+
 |  FIN    |<-----------------           ------------------>|  CLOSE  |
 | WAIT-1  |------------------                              |   WAIT  |
 +---------+          rcv FIN  \                            +---------+
   | rcv ACK of FIN   -------   |                            CLOSE  |
   | --------------   snd ACK   |                           ------- |
   V        x                   V                           snd FIN V
 +---------+                  +---------+                   +---------+
 |FINWAIT-2|                  | CLOSING |                   | LAST-ACK|
 +---------+                  +---------+                   +---------+
   |                rcv ACK of FIN |                 rcv ACK of FIN |
   |  rcv FIN       -------------- |    Timeout=2MSL -------------- |
   |  -------              x       V    ------------        x       V
    \ snd ACK                 +---------+delete TCB         +---------+
     ------------------------>|TIME WAIT|------------------>| CLOSED  |
                              +---------+                   +---------+
```

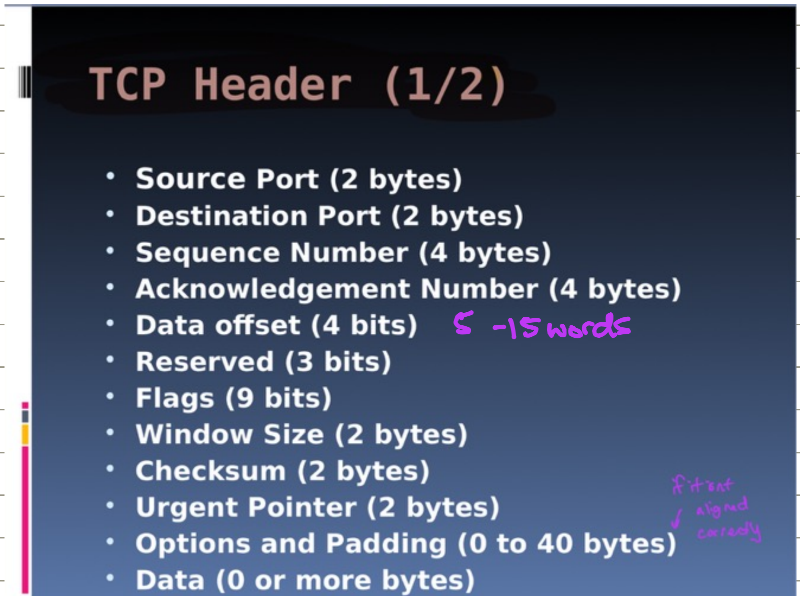
syn → syn+Ack → Ack

Mqtt con



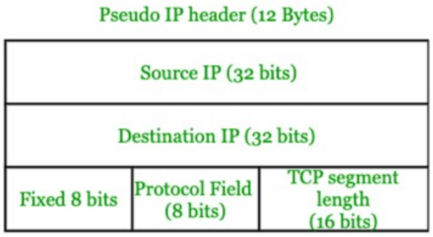
UDP Header

- Source Port Number (2 bytes)
- Destination Port Number (2 bytes)
  - Port numbers are used to send the data to the correct process (common ports are 21 FTP, 23 Telnet, 80 HTTP, 110 POP3)
- UDP Length (2 bytes)
  - Total length of UDP header and UDP data in bytes
- UDP Checksum (2 bytes)
  - Calculated on pseudo-header, UDP header, and UDP data
  - One's compliment sum of the preceding header, interpreted as 2 byte values
  - If byte count is odd, append a zero to the end



TCP Header (1/2)

- Source Port (2 bytes)
- Destination Port (2 bytes)
- Sequence Number (4 bytes)
- Acknowledgement Number (4 bytes)
- Data offset (4 bits) 5 -15 words
- Reserved (3 bits)
- Flags (9 bits)
- Window Size (2 bytes)
- Checksum (2 bytes)
- Urgent Pointer (2 bytes)
- Options and Padding (0 to 40 bytes) if it isnt ✓ aligned correctly
- Data (0 or more bytes)

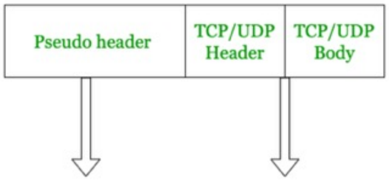


An important concept should be noted that this Pseudo header is created in the Transport layer for calculation and after the calculation is done the Pseudo header is **discarded**. And the checksum is calculated by using the usual checksum method.

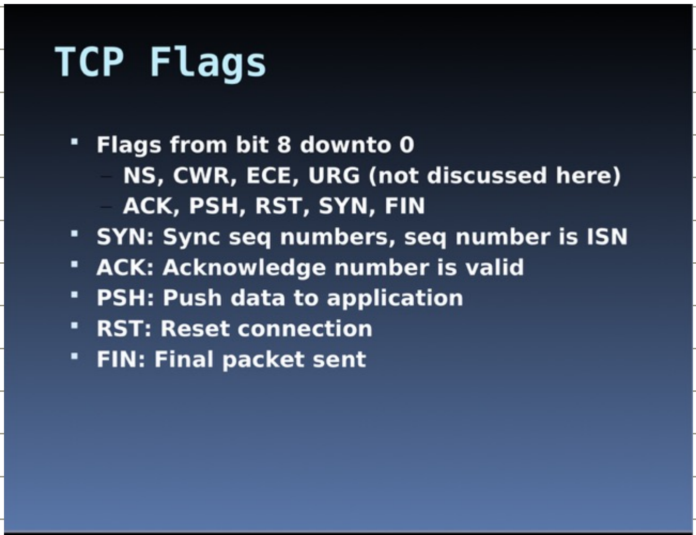
So, this Pseudo Header is not transported across the network, rather the actual IP header which is formed in Network Layer is transported.

The TCP checksum includes the following:

```
1. Pseudo IP header
2. TCP header
3. TCP body
```

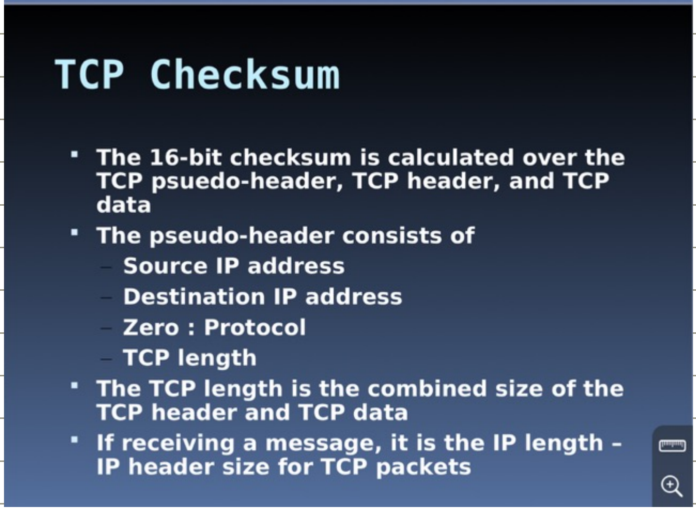


TCP/UDP CheckSum = Pseudo header + TCP/UDP Header + TCP/UDP Body



TCP Flags

- Flags from bit 8 downto 0
  - NS, CWR, ECE, URG (not discussed here)
  - ACK, PSH, RST, SYN, FIN
- SYN: Sync seq numbers, seq number is ISN
- ACK: Acknowledge number is valid
- PSH: Push data to application
- RST: Reset connection
- FIN: Final packet sent



TCP Checksum

- The 16-bit checksum is calculated over the TCP psuedo-header, TCP header, and TCP data
- The pseudo-header consists of
  - Source IP address
  - Destination IP address
  - Zero : Protocol
  - TCP length
- The TCP length is the combined size of the TCP header and TCP data
- If receiving a message, it is the IP length – IP header size for TCP packets